

Sorting Algorithms

1. Insertion Sort

e.g.	2	17	38	12	5	30	9	15	Comparisons
$i=0$	2								0
$i=1$	2	17	(2 < 17)						1
$i=2$	2	17	38	(17 < 38)					1
$i=3$	2	12	17	38	(12 < 38, 12 < 17, 12 > 2)				3
$i=4$	2	5	12	17	38				4
$i=5$	2	5	12	17	30	38			2
$i=6$	2	5	9	12	17	30	38		5
$i=7$	2	5	9	12	15	17	30	38	4
									20

Input size: n

How many steps to sort n elements?

Implement Insertion Sort:

```
for(int i=0 ; i < array.size() ; ++i) {
```

```
    for(int j=i ; j >= 1 ; --j) {
```

```
        if (array[j] < array[j-1]) {
```

```
            // swap array[j] & array[j-1]
```

```
        } else {
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

Time Complexity

best case: 1 2 3 4 5 6 7 ... n

i=0 1

i=1 1 2

i=2 1 2 3

i=3 1 2 3 4

i=4 1 2 3 4 5

How many comparisons?

$O(?)$

Worst Case:

$n \quad n-1 \quad n-2 \quad n-3 \quad \dots \quad 3, 2, 1$

$i=0 \quad n$

How many comparisons?

$i=1 \quad n-1 \quad n$

$i=0 : 0$

$i=2 \quad n-2 \quad n-1 \quad n$

$i=1 : 1$

$i=3 \quad n-3 \quad n-2 \quad n-1 \quad n$

$i=2 : 2$

$\vdots$

$i=3 : 3$

$i=4 : 4$

$$\text{total} = 0 + 1 + 2 + 3 + \dots + n-1$$

$$= \frac{(n-1) \cdot n}{2} = O(?)$$

In Summary:

Insertion sort

best Case

$O(n)$

Worst Case

$O(n^2)$

Divide and Conquer

In Computer Science, divide and conquer is an algorithm design paradigm based on recursion.

A divide and conquer algorithm breaks down a problem into two or more non-overlapping sub-problems of the same type, then recursively solve each sub-problem.

Quick Sort

e.g. 2 17 38 12 5 30 9 15 Comparisons

use last element as
pivot
↓

2 12 5 9 15 17 38 30 7

2 5 9 12 17 30 38 5

2 5

1

13

instance: a list A of n numbers

problem: sort A in ascending order

Quick Sort

divide: pick the last element of A as the pivot and
divide A into two lists A_1 and A_2 such that,
every element in $A_1 \leq \text{pivot}$
every element in $A_2 > \text{pivot}$

conquer: recursively sort A_1 & A_2 and get two
sorted lists A'_1 & A'_2 correspondingly.

recursion ends when the size of the sub-list is 1.

combine: combine A'_1 , pivot, A'_2 in this order
into one list A' .

return A' .

eg. 44 21 0 32 17 3 9 20

0 17 3 9 20 44 21 32

0 3 9 17 21 32 44

0 3

eg. 5 4 3 2 1

1 5 4 3 2

2 5 4 3

~~3 5 4~~

3 5 4

4 5

Time Complexity

Worst case:

1	2	3	4	...	n	Comparisons
1	2	3	...	<u>n-1</u>	<u>n</u>	n-1
1	2	3	...	<u>n-2</u>	<u>n-1</u>	n-2
1	2	3	...	<u>n-3</u>	<u>n-2</u>	n-3
						⋮
						1

$$\text{total} = 1 + 2 + 3 + \dots + n-1 = \frac{n(n-1)}{2} = O(?)$$

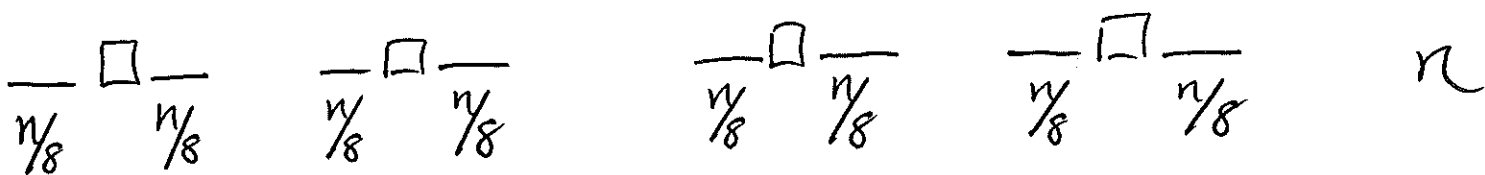
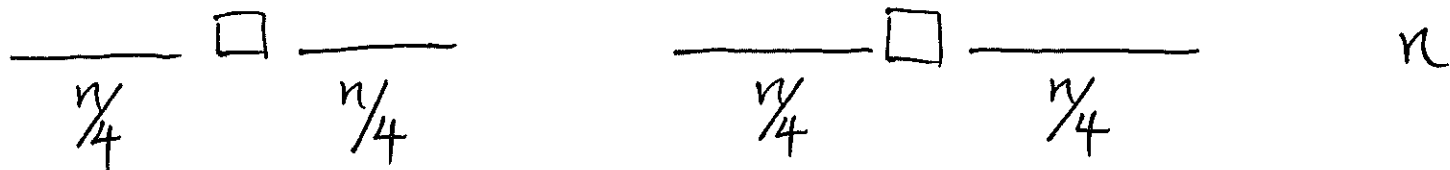
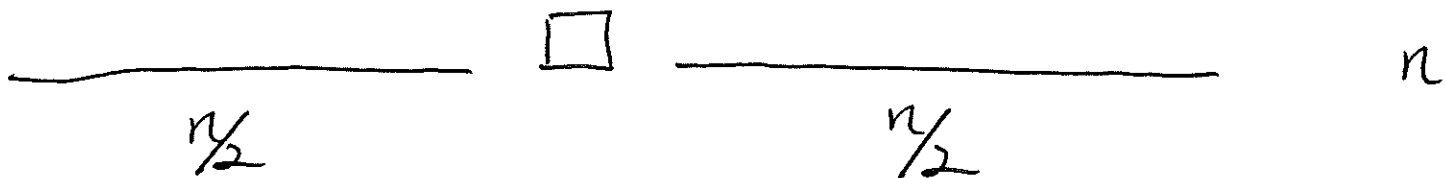
To avoid worst case,

we can pick the first, middle, and last element,
then find the medium of the three as the
pivot.

But not guarantee it works.

Best Case:

Comparisons



...

Recursion ends at ?

How many levels ?

8 elements

3 levels

e.g. 0000 | 0000

00 | 00 00 | 00

0/0 0/0 0/0 0/0

16 elements

0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0

0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0

0 0 | 0 0 0 0 | 0 0 0 0 | 0 0 0 0 | 0 0

0 | 0 0 | 0 0 | 0 0 | 0 0 | 0 0 | 0 0 | 0

4 levels

n elements — $\log n$ levels

each level : n comparisons

total : $O(n \cdot \log n)$

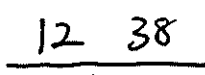
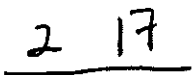
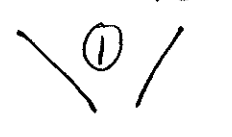
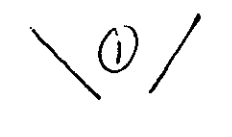
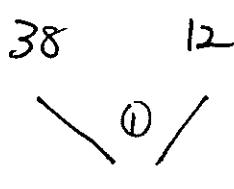
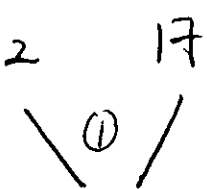
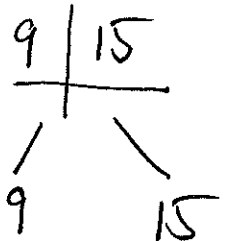
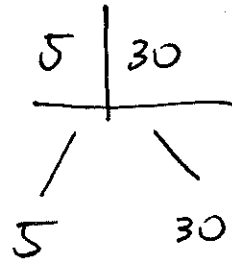
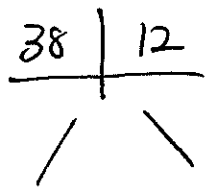
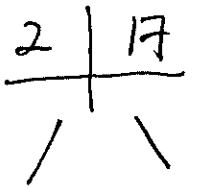
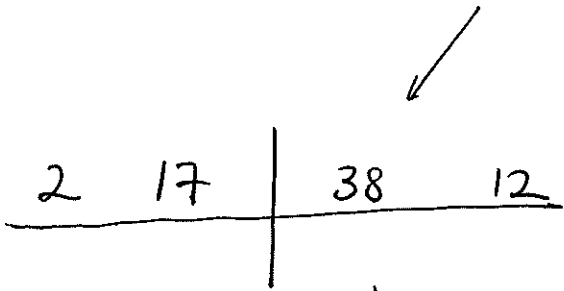
Summary : Quick Sort

worst case $O(n^2)$

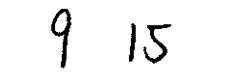
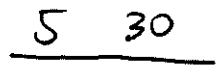
best case $O(n \cdot \log n)$

Merge Sort

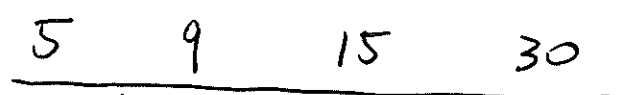
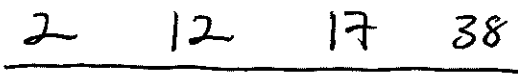
eg. 2 17 38 12 | 5 30 9 15



③



③



⑦

2 5 9 12 15 17 30 38

instance: An array $A[1, 2, \dots, n]$ of n numbers

problem: Sort A in ascending order

Merge Sort

Divide: Divide A into two roughly equal halves,

$$A_1[1, 2, \dots, n/2]$$

$$A_2[n/2+1, \dots, n]$$

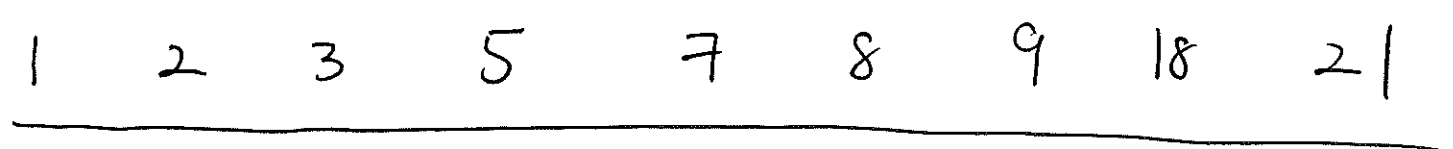
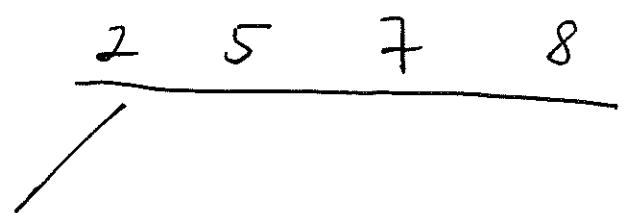
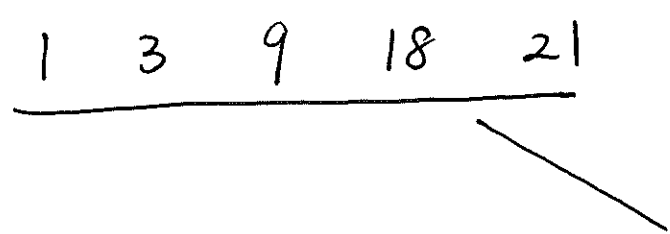
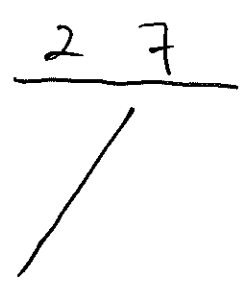
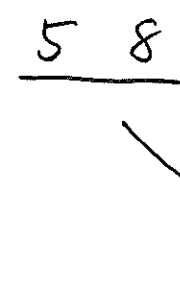
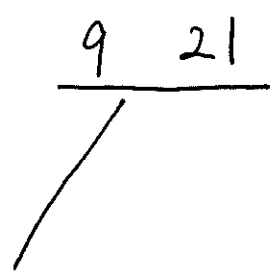
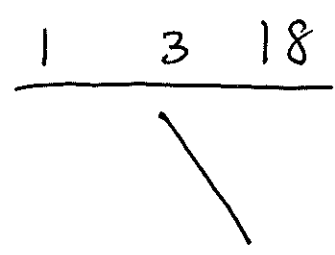
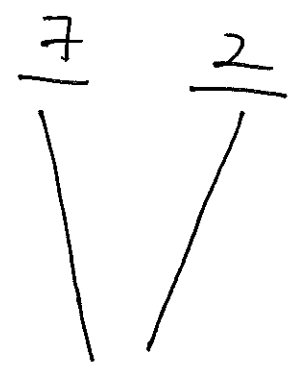
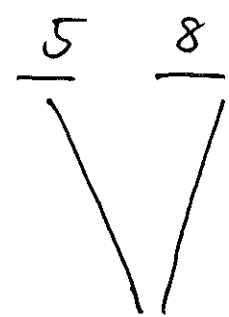
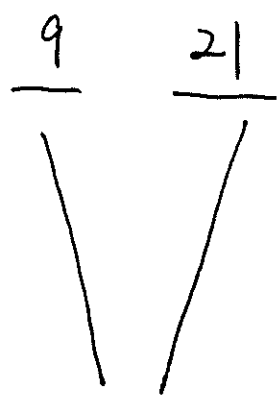
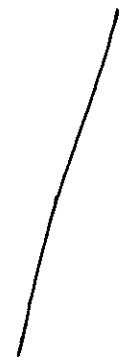
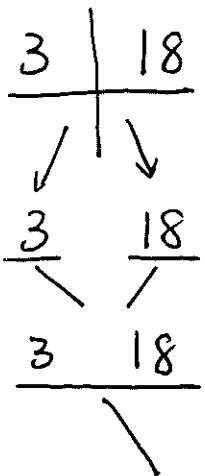
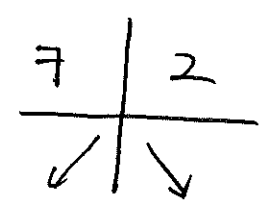
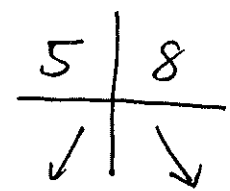
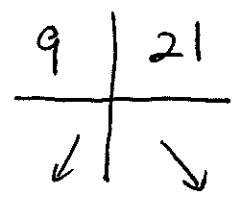
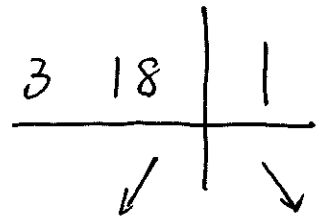
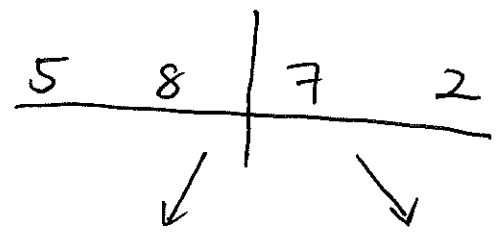
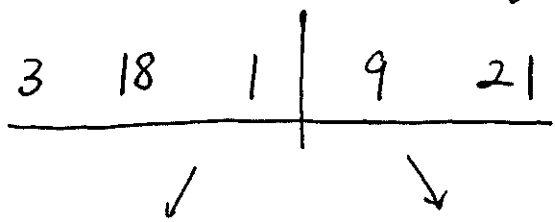
Conquer: recursively sort A_1 and A_2 and get two sorted arrays A'_1 and A'_2 , respectively.

Recursion ends when the size of A_1 or A_2 is 1.

Combine: Merge two sorted arrays A'_1 and A'_2 into one array A' .

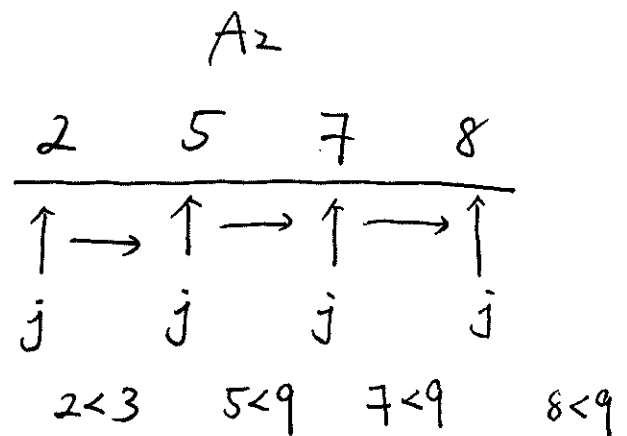
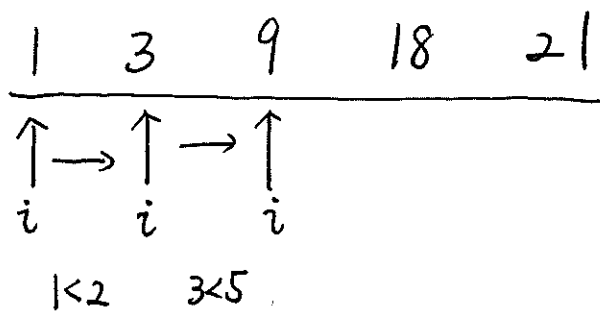
return A'

eg. 3 18 1 9 21 | 5 8 7 2



key to merge sort:

Merge two sorted lists into one



If the size of $A_1 + A_2$ is n ,

what is the running time to merge A_1 & A_2 ?

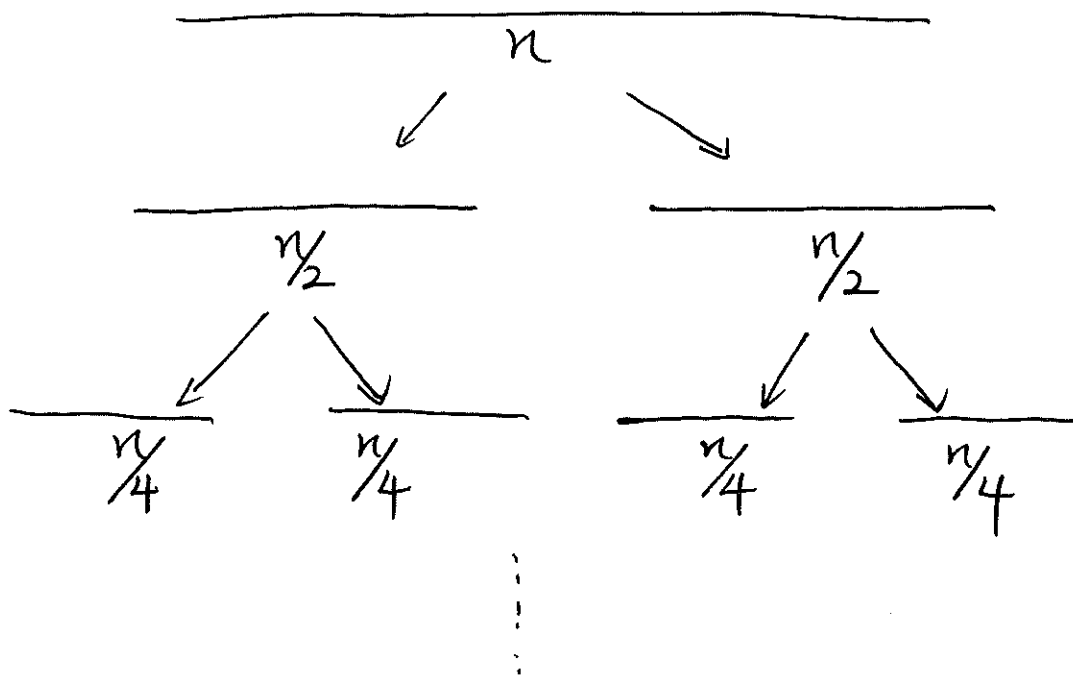
$O(?)$

Time Complexity of merge sort:

Divide n into two roughly equal halves.

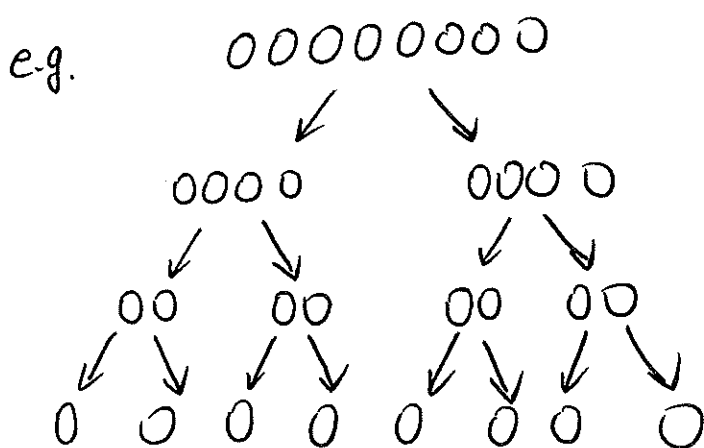
recursively divide

recursion ends when ~~size~~ size = 1



How many levels?

$O(?)$



each level, there are n elements to merge.

so each level: $O(n)$

totally: $O(n \cdot \log n)$